CE-2103 · Lección 08

Bases de Datos

De los archivos a SQL y NoSQL

bases_de_datos.sql

```sql
-- Lección 08
SELECT *
FROM bases_de_datos
WHERE tema IN ('SQL', 'NoSQL');
```

Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

// introducción

¿Qué aprenderemos hoy?

1. Qué es un DBMS y por qué reemplazó a los archivos
2. Cómo se modelan los datos en el modelo relacional
3. Cómo los índices (árboles B+) aceleran las búsquedas
4. Cómo consultar y modificar datos con SQL, también desde Python y Java
5. Cómo diseñar una base de datos relacional paso a paso
6. Qué ofrecen las bases de datos NoSQL y cuándo usarlas

// introducción

Bases de datos en la vida diaria

Usted usa bases de datos todos los días, casi sin notarlo:

Banco

Depositar o retirar fondos y consultar el saldo.

Cuentas · saldos

Reservas

Reservar un hotel o un vuelo y ver la disponibilidad.

Habitaciones · vuelos

Biblioteca

Buscar un libro en el catálogo computarizado.

Libros · préstamos

Compra en línea

Elegir productos y pagar. Es el caso que seguiremos en la lección.

Clientes · pedidos

Propósito: toda base de datos se **diseña, construye y puebla con datos** para un fin concreto, pensando en sus **usuarios y aplicaciones previstas**.

// fundamentos

De un dato a un DBMS

1

Dato

Representación simbólica de un hecho o atributo. Ej.: la edad de una persona o el nombre de un producto.

2

Base de datos

Colección de datos organizada para un propósito específico, dentro de un dominio concreto.

3

DBMS

Software que permite crear, leer, actualizar y eliminar datos de una base de datos.

Recuerde: no hay bases de datos genéricas. Cada una describe un dominio (universo de discurso) específico.

// fundamentos

Algunos DBMS populares

Cada DBMS adopta un enfoque: **relacional** (verde azulado) o **NoSQL** (ámbar).

MySQL
Relacional

PostgreSQL
Objeto-relacional

MariaDB
Relacional

MongoDB
NoSQL: documentos

SQLite
Relacional (embebido)

CockroachDB
Relacional distribuido

Redis
NoSQL: clave-valor

CouchDB
NoSQL: documentos

Neo4j
NoSQL: grafos

FirebirdSQL
Relacional

OrientDB
NoSQL: multimodelo

Cassandra
NoSQL: columnas

// enfoques de manejo de datos

Antes de los DBMS: archivos

Cada programa leía y escribía sus propios archivos. Al crecer el sistema aparecen problemas:

Formatos inconsistentes

Cada archivo define su propio formato.

Redundancia

Los mismos datos duplicados en archivos separados.

Diseño ineficiente

Estructuras pensadas para un solo programa.

Inseguridad

Poco control sobre quién accede a los datos.

Acoplamiento alto

Cambiar un archivo obliga a cambiar los programas.

Integridad frágil

Un error en un programa puede corromper los datos.

Enfoque de bases de datos: una capa de indirección



Independencia de los datos: los programas no necesitan conocer la estructura interna. El DBMS es auto-descriptivo (guarda metadatos).

Ventajas del enfoque de bases de datos

Independencia de datos

Los programas no dependen de la estructura interna de los datos.

Integridad

Reglas que garantizan datos precisos y consistentes.

Seguridad

Control de acceso y protección contra accesos no autorizados.

Concurrencia

Acceso simultáneo de múltiples usuarios sin conflictos.

Escalabilidad

Maneja grandes volúmenes de datos y crece con la organización.

Datos compartidos

Varias aplicaciones usan los mismos datos sin duplicarlos.

// tipos de bases de datos

Cada base de datos adopta un enfoque

Como los lenguajes de programación con sus paradigmas, las bases de datos siguen un enfoque:

1

Relacional

Tablas de filas y columnas relacionadas por claves. Modelo de Edgar Codd (1970).

2

Orientada a objetos

Los datos se modelan como objetos con atributos y métodos.

3

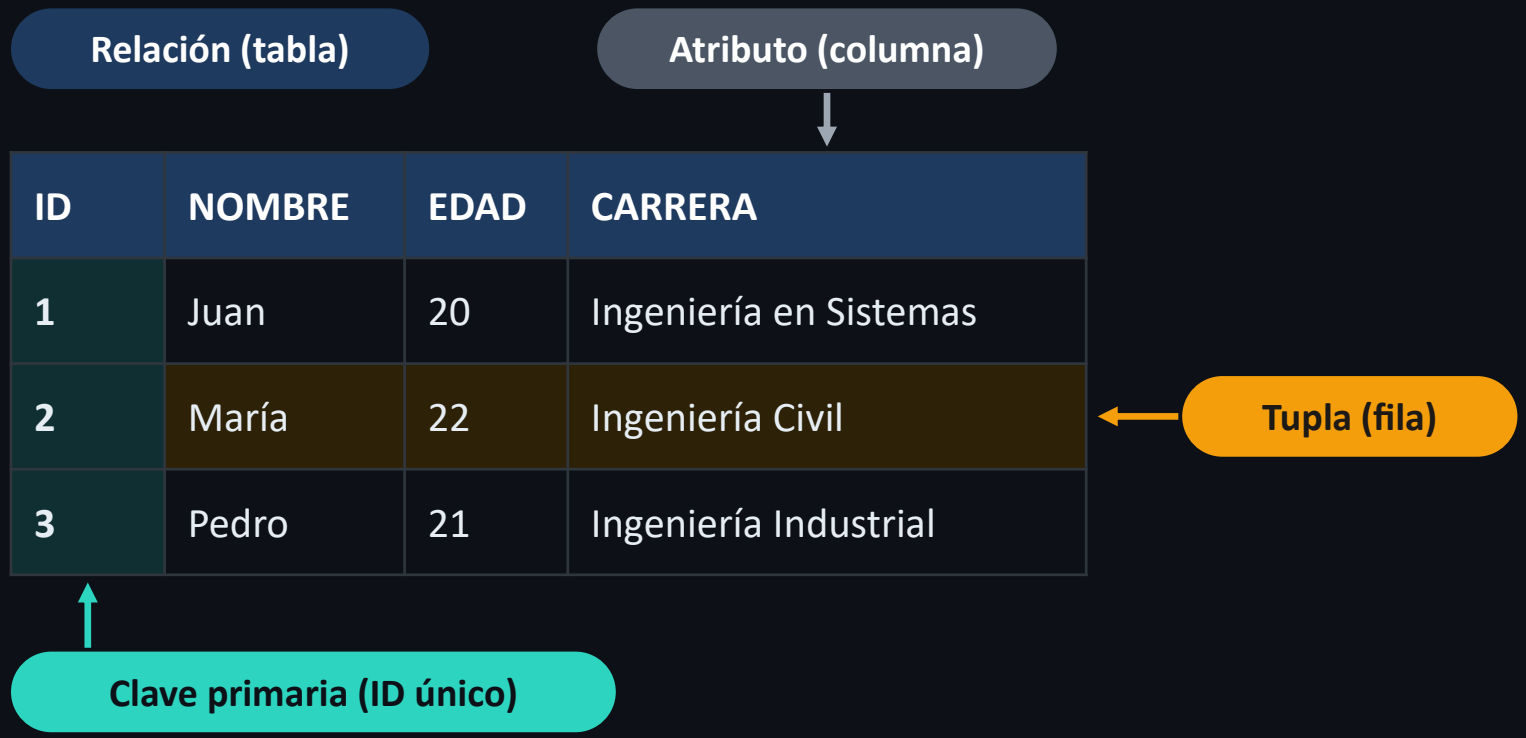
NoSQL

Alternativa flexible y escalable para datos no estructurados o semiestructurados.

> Un DBMS puede soportar más de un enfoque (p. ej., Oracle: relacional y objetos). Por eso decimos que la **base de datos** sigue un enfoque, no el DBMS.

// modelo relacional

Anatomía de una tabla



Otros términos

Esquema: estructura de la tabla (como una clase en POO).

Clave foránea: referencia a la clave primaria de otra tabla.

Clave candidata: atributo que podría ser clave primaria.

// modelo relacional

Relaciones muchos-a-muchos

ESTUDIANTE

ID	NOMBRE
1	Juan
2	María
3	Pedro

FK

MATRICULA (tabla intermedia)

ESTUDIANTE_ID	CURSO_ID
1	1
1	2
2	1

FK

CURSO

ID	NOMBRE
1	Matemáticas
2	Física
3	Química

PK clave primaria **FK clave foránea**

Un estudiante matricula **muchos** cursos y un curso tiene **muchos** estudiantes. La **tabla intermedia** guarda solo las claves (sin duplicar datos del estudiante ni del curso).

Integridad referencial y normalización

Integridad referencial

Restricción que valida que cada clave foránea apunte a una clave primaria existente.

Ej.: `MATRICULA.ESTUDIANTE_ID` debe existir en `ESTUDIANTE.ID`.

vs

Normalización

Proceso de diseño del esquema, basado en las formas normales, que reduce la redundancia de datos.

Se aplica al diseñar las tablas.

Ojo: la normalización **no** es consecuencia de la integridad referencial. Son conceptos distintos que se complementan.

Transacciones ACID

Una transacción es una operación que modifica los datos y debe cumplir cuatro propiedades:

A

Atomicidad

Todo o nada: si una parte falla, se deshace la transacción completa.

C

Consistencia

Lleva la base de un estado consistente a otro, respetando las restricciones.

I

Aislamiento

Las transacciones concurrentes no interfieren entre sí.

D

Durabilidad

Los cambios confirmados persisten aun después de un fallo del sistema.

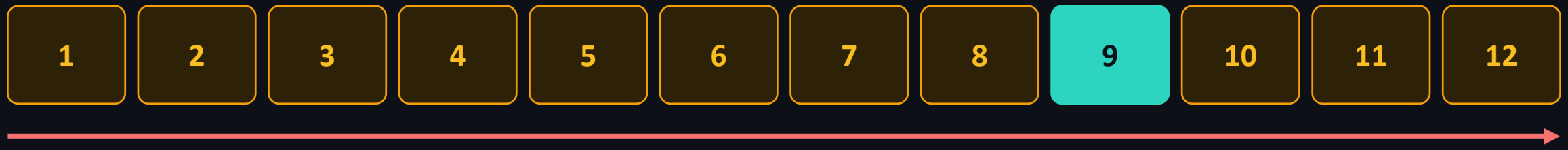
El problema: full table scan

```sql
SELECT *
FROM ESTUDIANTE
WHERE NOMBRE = 'María';
```

O(n)

costo del recorrido completo

Tabla ESTUDIANTE con n filas



El DBMS revisa **todas** las filas, una por una, comparando `NOMBRE` con 'María'.

> Sin índice, el DBMS no sabe dónde están las filas que cumplen la condición. Con millones de registros resulta prohibitivamente lento.

¿Qué es un índice?

Estructura auxiliar que asocia el valor de una columna (la **clave**) con la ubicación de sus filas.

Índice (por NOMBRE)

NOMBRE	Apunta a
Juan	fila 1
María	fila 2
Pedro	fila 3

Tabla ESTUDIANTE

ID	NOMBRE	EDAD
1	Juan	20
2	María	22
3	Pedro	21

> **Analogía:** el índice de un libro. En vez de leer todas las páginas, busca el término y va directo a la página.

Árbol B+

Nodos con muchas claves: árbol poco profundo y pocos accesos a disco. Sus hojas enlazadas permiten recorrer rangos.

Costo de búsqueda

`O(log n)` en vez de `O(n)`.

Árbol B+ vs. índice hash

Árbol B+

- Igualdad en `O(log n)`
- Soporta rangos: `WHERE EDAD > 20`
- Permite recorridos ordenados
- Predeterminado en la mayoría de DBMS

vs

Hash

- Igualdad exacta en `O(1)` promedio
- No soporta rangos
- No preserva el orden de las claves
- Útil solo para búsquedas exactas

El árbol B+ es la misma familia de estructuras que estudiamos en almacenamiento externo.

Clustered vs. non-clustered

	Clustered (agrupado)	Non-clustered (secundario)
Almacenamiento	La tabla se ordena físicamente según la clave; las filas son las hojas del árbol.	Estructura separada con las claves ordenadas y un puntero a la fila real.
Acceso al dato	Directo, sin salto adicional.	Un salto extra hasta la fila.
Cantidad por tabla	Uno solo.	Varios.

> Al declarar una **clave primaria**, el DBMS suele crear un índice sobre ella (frecuentemente clustered).

No conviene indexar todo

Lecturas más rápidas

Evitan el full table scan en consultas con `WHERE`, `JOIN` u `ORDER BY`.

Escrituras más lentas

Cada `INSERT`, `UPDATE` o `DELETE` también actualiza los índices afectados.

Más espacio en disco

Cada índice es una estructura aparte que ocupa almacenamiento.

Regla práctica: indexe columnas de uso frecuente en `WHERE`, `JOIN` u `ORDER BY`. Evítelos en tablas con muchas escrituras o columnas que casi no se consultan.

// índices

Crear un índice en SQL

Crear el índice

```sql
CREATE INDEX idx_estudiante_nombre
ON ESTUDIANTE (NOMBRE);
```

La misma consulta de antes

```sql
SELECT *
FROM ESTUDIANTE
WHERE NOMBRE = 'María';
```

Sin índice

Full table scan: revisa las n filas.

O(n)

Con índice

Búsqueda en el árbol B+ del índice.

O(log n)

> La diferencia es despreciable en tablas pequeñas, pero determinante cuando n es grande.

// sql

SQL: DDL y DML

DDL · Data Definition Language

Define y modifica las **estructuras** de la base de datos.

CREATE ALTER DROP

DML · Data Manipulation Language

Manipula los **datos** dentro de esas estructuras.

SELECT INSERT UPDATE DELETE

> SQL (Structured Query Language) es el lenguaje estandarizado para interactuar con bases de datos relacionales.

// sql

DDL: CREATE TABLE

ESTUDIANTE

```sql
CREATE TABLE ESTUDIANTE (
  ID INT PRIMARY KEY,
  NOMBRE VARCHAR(50),
  EDAD INT,
  CARRERA VARCHAR(50)
);
```

CURSO

```sql
CREATE TABLE CURSO (
  ID INT PRIMARY KEY,
  NOMBRE VARCHAR(50)
);
```

MATRICULA (tabla intermedia)

```sql
CREATE TABLE MATRICULA (
  ESTUDIANTE_ID INT,
  CURSO_ID INT,
  PRIMARY KEY (ESTUDIANTE_ID, CURSO_ID),
  FOREIGN KEY (ESTUDIANTE_ID)
    REFERENCES ESTUDIANTE(ID),
  FOREIGN KEY (CURSO_ID)
    REFERENCES CURSO(ID)
);
```

> Los tipos de datos varían según el DBMS. Las restricciones también pueden agregarse después con `ALTER TABLE`.

// sql

DML: INSERT, UPDATE y DELETE

INSERT · agregar filas

```sql
INSERT INTO CURSO (ID, NOMBRE)
VALUES (1, 'Matemáticas');
```

UPDATE · modificar filas

```sql
UPDATE ESTUDIANTE
SET EDAD = 21
WHERE ID = 1;
```

DELETE · eliminar filas

```sql
DELETE FROM CURSO
WHERE ID = 1;
```

Cuidado: sin `WHERE`, `UPDATE` y `DELETE` afectan **todas** las filas de la tabla.

Integridad: al borrar en una tabla «padre», los registros «hijos» se eliminan o actualizan (acción en cascada).

// sql

DML: SELECT

Estructura

```sql
SELECT columna1, columna2
FROM tabla
WHERE condición;
```

Todos los estudiantes

```sql
SELECT *
FROM ESTUDIANTE;
```

Cursos de un estudiante específico

```sql
SELECT C.*
FROM CURSO C
JOIN MATRICULA M ON C.ID = M.CURSO_ID
WHERE M.ESTUDIANTE_ID = 1;
```

> * significa «todas las columnas». C.* son todas las columnas de CURSO (alias C).

// sql

JOIN: combinar tablas

Consulta

```sql
SELECT E.NOMBRE, C.NOMBRE
FROM ESTUDIANTE E
JOIN MATRICULA M
    ON E.ID = M.ESTUDIANTE_ID
JOIN CURSO C
    ON C.ID = M.CURSO_ID;
```

Resultado

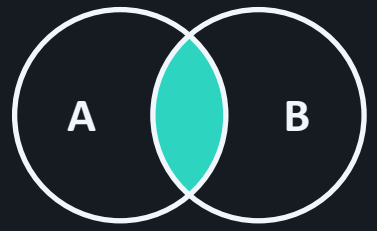
E.NOMBRE	C.NOMBRE
Juan	Matemáticas
Juan	Física
María	Matemáticas

La tabla intermedia `MATRICULA` conecta cada estudiante con sus cursos. Cada fila de `MATRICULA` produce una fila en el resultado.

// sql

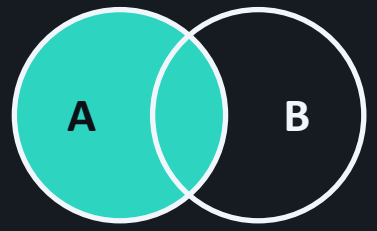
Tipos de JOIN

Combinan filas de dos tablas (A y B) según una condición `ON`. El área coloreada es lo que se devuelve.



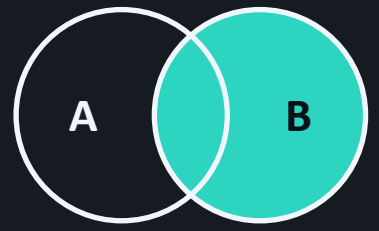
INNER JOIN

Solo las filas con coincidencia en ambas tablas.



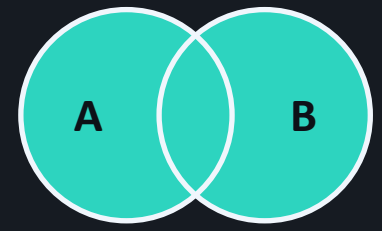
LEFT OUTER JOIN

Todas las filas de A, más las coincidencias de B.



RIGHT OUTER JOIN

Todas las filas de B, más las coincidencias de A.



FULL OUTER JOIN

Todas las filas de A y de B, coincidan o no.

> `OUTER` es opcional: `LEFT JOIN` equivale a `LEFT OUTER JOIN`. Y `JOIN` a secas equivale a `INNER JOIN`.

// sql

LEFT OUTER JOIN en acción

Consulta

```sql
SELECT E.NOMBRE, C.NOMBRE
FROM ESTUDIANTE E
LEFT OUTER JOIN MATRICULA M
    ON E.ID = M.ESTUDIANTE_ID
LEFT OUTER JOIN CURSO C
    ON C.ID = M.CURSO_ID;
```

Resultado

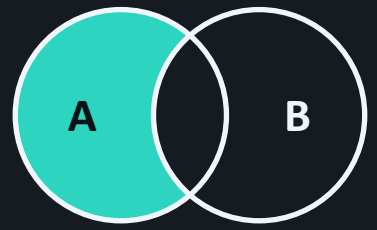
E.NOMBRE	C.NOMBRE
Juan	Matemáticas
Juan	Física
María	Matemáticas
Pedro	NULL

Ojo: Pedro no tiene matrículas, pero **sí aparece**, con `NULL` en el curso. Con `INNER JOIN` desaparecería.

// sql

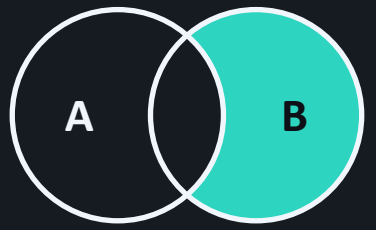
JOIN para encontrar filas sin pareja

Para quedarse solo con lo que **no** coincide, se agrega `WHERE ... IS NULL` sobre la clave de la otra tabla.



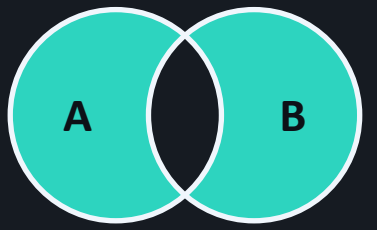
Solo A

`LEFT JOIN` y `WHERE` clave de B `IS NULL`. Ej.: estudiantes sin matrícula.



Solo B

`RIGHT JOIN` y `WHERE` clave de A `IS NULL`. Ej.: cursos sin estudiantes.



A o B, no ambas

`FULL OUTER JOIN` y `WHERE` clave de A `IS NULL` o de B `IS NULL`.

Ej.: `LEFT JOIN MATRICULA M … WHERE M.ESTUDIANTE_ID IS NULL` devuelve solo a **Pedro**.

// sql

CROSS JOIN y SELF JOIN

CROSS JOIN

Combina **cada fila** de A con **cada fila** de B, sin condición `ON`.

```sql
SELECT E.NOMBRE, C.NOMBRE
FROM ESTUDIANTE E
CROSS JOIN CURSO C;
```

*3 estudiantes × 3 cursos = **9 filas**.*

SELF JOIN

Una tabla se une **consigo misma**, con dos alias. Tabla: `EMPLEADO(ID, NOMBRE, JEFE_ID)`.

```sql
SELECT E.NOMBRE AS EMPLEADO,
       J.NOMBRE AS JEFE
FROM EMPLEADO E
JOIN EMPLEADO J
    ON E.JEFE_ID = J.ID;
```

Cuidado: `CROSS JOIN` crece como n × m: con 10 000 × 10 000 filas son 100 millones.

// desde el código

Conectar una aplicación a la base de datos

1

Conectar

Abrir la conexión con la URL, el usuario y el driver.

2

Preparar

Escribir el SQL con parámetros `?`.

3

Ejecutar

El DBMS devuelve filas o cuántas cambió.

4

Recorrer

Leer cada fila del resultado y cerrar la conexión.

Driver: **Python** usa la API DB-API (`sqlite3`, `psycopg2`…); **Java** usa **JDBC** (`java.sql`). El patrón es el mismo con cualquier DBMS.

// desde el código

Python: sqlite3 y la DB-API

Conectar y consultar

```
import sqlite3

con = sqlite3.connect("universidad.db")
cur = con.cursor()
cur.execute(
    "SELECT NOMBRE, EDAD FROM ESTUDIANTE WHERE ID = ?",
    (1,))
for nombre, edad in cur.fetchall():
    print(nombre, edad)
con.close()
```

Insertar y confirmar

```
cur.execute(
    "INSERT INTO CURSO "
    "VALUES (?, ?)",
    (4, "Programación"))
con.commit()
```

Salida de la consulta

```
Juan 20
```

Cada `?` es un parámetro: el valor viaja aparte, en la tupla `(1,)`. Sin `commit()`, los cambios no se guardan.

// desde el código

Java: JDBC

Conectar y consultar

```
String url = "jdbc:sqlite:universidad.db";
String sql = "SELECT NOMBRE, EDAD FROM ESTUDIANTE WHERE ID = ?";
try (Connection con = DriverManager.getConnection(url);
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setInt(1, 1);
    ResultSet rs = ps.executeQuery();
    while (rs.next()) {
        String nombre = rs.getString("NOMBRE");
        int edad = rs.getInt("EDAD");
        System.out.println(nombre + " " + edad);
    }
}
```

Insertar

```
String ins = "INSERT INTO CURSO "
    + "(ID, NOMBRE) VALUES (?, ?)";
try (PreparedStatement p =
          con.prepareStatement(ins)) {
    p.setInt(1, 4);
    p.setString(2, "Programación");
    p.executeUpdate();
}
```

`try-with-resources` cierra la conexión por usted. JDBC confirma cada sentencia sola (autocommit); en Python hay que llamar a `commit()`.

// desde el código

Nunca pegue la entrada del usuario en el SQL

Concatenar (inseguro)

Se arma el SQL pegando lo que escribe el usuario.

```
nombre = input("Nombre: ")
cur.execute(
    "SELECT * FROM ESTUDIANTE "
    "WHERE NOMBRE = '" + nombre + "'")
```

*Con `x' OR '1'='1` devuelve **todas** las filas.*

vs

Parametrizar (seguro)

El SQL lleva un `?` y el valor se envía aparte.

```
nombre = input("Nombre: ")
cur.execute(
    "SELECT * FROM ESTUDIANTE "
    "WHERE NOMBRE = ?", (nombre,))
```

La misma entrada no encuentra a nadie: es solo texto.

SQL injection: es uno de los ataques más comunes. En Java use siempre `PreparedStatement` con `?`, nunca `Statement` con `+`.

// desde el código

Buenas prácticas al conectarse

Cierre los recursos

Cierre siempre la conexión: `try-with-resources` en Java, `close()` en Python.

Credenciales fuera del código

Usuario, contraseña y URL van en variables de entorno o en configuración, nunca en el repositorio.

Use transacciones

Agrupe operaciones relacionadas y confirme con `commit()` o deshaga con `rollback()` (ACID).

Pool de conexiones y ORM

Abrir conexiones cuesta: se reutilizan con un pool. Un ORM mapea tablas a objetos.

// diseño relacional

Una sola tabla: ¿qué puede salir mal?

PEDIDOS (todo junto)

PEDIDO	FECHA	CLIENTE	CORREO	PRODUCTO	PRECIO	CANTIDAD
1	03/11	Ana	ana@mail.com	Teclado	25	1
1	03/11	Ana	ana@mail.com	Mouse	10	2
2	04/11	Ana	ana@mail.com	Mouse	10	1
3	04/11	Luis	luis@mail.com	Teclado	25	1

Actualizar

Cambiar el correo de Ana obliga a editar 3 filas.

Insertar

No se puede registrar un cliente que aún no tiene pedidos.

Eliminar

Al borrar el pedido 3 se pierde también a Luis.

// diseño relacional

Diseñar una base de datos en 6 pasos

Caso que usaremos de principio a fin: **una tienda en línea**.

1 · Requisitos

¿Qué guarda el sistema y qué debe responder?

2 · Entidades

Las «cosas» importantes y sus atributos.

3 · Relaciones

¿Cómo se conectan? 1:1, 1:N o N:M.

4 · Tablas y claves

Del modelo a tablas con claves primarias y foráneas.

5 · Normalizar

Elimine la redundancia con las formas normales.

6 · Crear y probar

Escriba el SQL, cargue datos y pruebe consultas.

Del mini-mundo al disco: diseño **conceptual** (pasos 2-3, no depende del DBMS) → **lógico** (4-5, modelo de datos del DBMS) → **físico** (6, estructuras concretas, p. ej. índices).

Paso 1 · Requisitos de la tienda

Lo que dice el negocio

- Los **clientes** se registran con su nombre y su correo (único).
- Un cliente puede hacer **muchos pedidos**; cada pedido es de un solo cliente.
- Cada pedido tiene una **fecha** y varios **productos**.
- De cada producto se guarda el **nombre** y el **precio**.
- En cada pedido se anota la **cantidad** de cada producto.

Cómo leerlo

- **Sustantivos:** posibles entidades o atributos
- **Verbos:** posibles relaciones (hace, contiene)
- **Cuántos:** cardinalidad (1:1, 1:N, N:M)

*Ej.: «Un **cliente hace pedidos**» → entidades CLIENTE y PEDIDO, y la relación «hace».*

Paso 2 · Entidades y atributos

CLIENTE	
ID	único
NOMBRE	
CORREO	

PEDIDO	
ID	único
FECHA	

PRODUCTO	
ID	único
NOMBRE	
PRECIO	

¿Y la cantidad?

No es del pedido ni del producto: depende de **ambos**. La resolveremos en el paso 3.

Entidad: una «cosa» sobre la que guardamos datos. **Atributo:** una propiedad de esa cosa. Toda entidad necesita un **identificador único** (`ID`).

Paso 3 · Relaciones y cardinalidad



1:N · uno a muchos

Un cliente hace **muchos** pedidos. Cada pedido pertenece a **un solo** cliente.

N:M · muchos a muchos

Un pedido contiene **varios** productos y un producto aparece en **varios** pedidos. La `CANTIDAD` es un atributo de la relación.

> Cardinalidades posibles: **1:1** (uno a uno), **1:N** (uno a muchos) y **N:M** (muchos a muchos).

Paso 4 · De relaciones a tablas

1:1

Uno a uno

Una clave foránea **única** en una de las dos tablas.

p. ej., USUARIO y PERFIL

1:N

Uno a muchos

La clave foránea va en la tabla del lado **«muchos»**.

PEDIDO.CLIENTE_ID

N:M

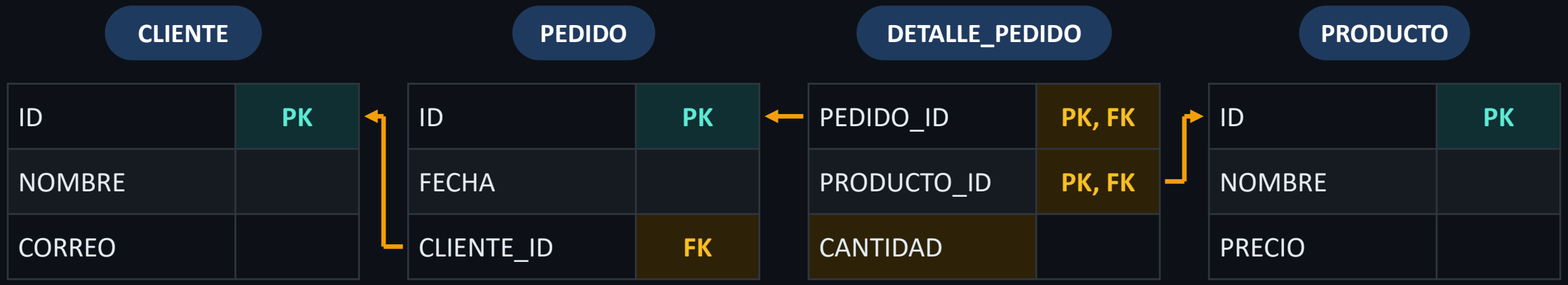
Muchos a muchos

Se crea una **tabla intermedia** con dos claves foráneas.

DETALLE_PEDIDO

Además: cada entidad se convierte en una **tabla** y su `ID` en la **clave primaria**.

Paso 4 · El esquema relacional



1:N: `PEDIDO.CLIENTE_ID` apunta a `CLIENTE.ID`. **N:M:** `DETALLE_PEDIDO` une pedidos y productos; su clave primaria es compuesta y `CANTIDAD` vive ahí.

Paso 5 · Revisar las formas normales

Regla de oro: cada dato vive **en un solo lugar**. Nuestro esquema cumple estas tres formas normales:

1FN

Un valor por celda

Mal: `PRODUCTOS = 'Teclado, Mouse'`. **Bien:** una fila por cada producto.

2FN

Toda la clave

Mal: `NOMBRE_PRODUCTO` dentro de DETALLE_PEDIDO (depende solo del producto). **Bien:** en PRODUCTO.

3FN

Nada más que la clave

Mal: `CORREO_CLIENTE` dentro de PEDIDO (depende del cliente). **Bien:** en CLIENTE.

Paso 6 · Crear las tablas

PEDIDO · relación 1:N

```
CREATE TABLE PEDIDO (
  ID INT PRIMARY KEY,
  FECHA DATE NOT NULL,
  CLIENTE_ID INT NOT NULL,
  FOREIGN KEY (CLIENTE_ID)
    REFERENCES CLIENTE(ID)
);
```

DETALLE_PEDIDO · relación N:M

```
CREATE TABLE DETALLE_PEDIDO (
  PEDIDO_ID INT,
  PRODUCTO_ID INT,
  CANTIDAD INT NOT NULL,
  PRIMARY KEY (PEDIDO_ID, PRODUCTO_ID),
  FOREIGN KEY (PEDIDO_ID)
    REFERENCES PEDIDO(ID),
  FOREIGN KEY (PRODUCTO_ID)
    REFERENCES PRODUCTO(ID)
);
```

Restricciones útiles

`NOT NULL` obliga a dar un valor. `UNIQUE` evita repetidos (p. ej., el correo).

Paso 6 · Probar con consultas

¿Qué compró cada cliente?

```
SELECT C.NOMBRE, P.NOMBRE, D.CANTIDAD
FROM CLIENTE C
JOIN PEDIDO O ON O.CLIENTE_ID = C.ID
JOIN DETALLE_PEDIDO D ON D.PEDIDO_ID = O.ID
JOIN PRODUCTO P ON P.ID = D.PRODUCTO_ID;
```

Resultado

C.NOMBRE	P.NOMBRE	D.CANTIDAD
Ana	Teclado	1
Ana	Mouse	2
Ana	Mouse	1
Luis	Teclado	1

El `JOIN` recorre el esquema siguiendo las claves foráneas y reconstruye la información de la tabla única, ahora sin redundancia.

// diseño relacional

Ahora usted: extienda el modelo

Aplique los 6 pasos a cada requisito nuevo: dibuje, defina tablas y claves, y escriba el SQL.

A

Categorías

Cada producto pertenece a una categoría. ¿Qué cardinalidad hay? ¿Dónde va la clave foránea?

B

Reseñas

Los clientes califican productos con estrellas y un comentario. ¿Qué tabla nueva aparece?

C

Precio histórico

El precio cambia con el tiempo. ¿Cómo conservar lo que pagó cada cliente?

// nosql

NoSQL: origen y características

1998

Primer uso del nombre

Carlo Strozzi llamó «NoSQL» a su base de datos ligera, que literalmente no usaba SQL.

2009

Nuevo significado

Se reinterpreta como «Not Only SQL»: existen alternativas al modelo relacional.

Estructura flexible

No requiere un esquema fijo.

Escalabilidad horizontal

Reparte la carga entre varios servidores o nodos.

Modelos diversos

Documentos, grafos, columnas y clave-valor.

// nosql

Tipos de bases de datos NoSQL

Documentos

Documentos JSON o BSON agrupados en colecciones (similares a tablas).

MongoDB · CouchDB

Grafos

Nodos y relaciones entre ellos. Ideales para relaciones complejas.

Neo4j · Neptune

Columnas

Datos por columnas en vez de filas. Eficientes en análisis y agregaciones.

Cassandra · HBase

Clave-valor

Pares clave-valor con claves únicas. Lecturas y escrituras muy rápidas.

Redis · DynamoDB

> Cada tipo está optimizado para un tipo distinto de aplicación y carga de trabajo.

// nosql

Una misma información, dos modelos

Relacional

- Datos repartidos en varias tablas
- Se reconstruyen con JOIN
- Esquema fijo y definido de antemano

```
SELECT E.NOMBRE, C.NOMBRE
FROM ESTUDIANTE E
JOIN MATRICULA M ON E.ID = M.ESTUDIANTE_ID
JOIN CURSO C ON C.ID = M.CURSO_ID;
```

vs

Documentos

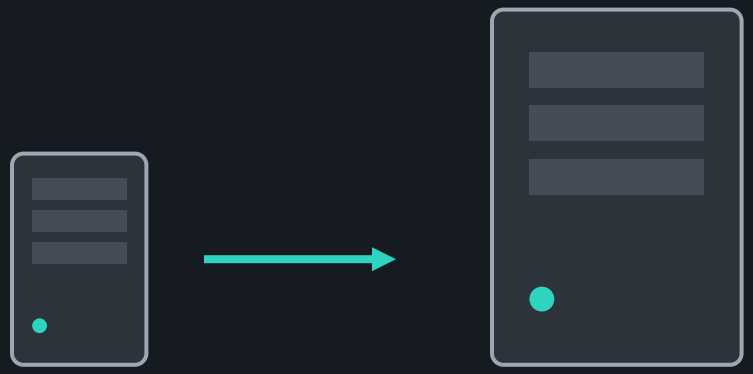
Todo en un solo documento; su estructura puede variar entre documentos.

```
{
  "id": 1,
  "nombre": "Juan",
  "edad": 20,
  "carrera": "Ingeniería en Sistemas",
  "cursos": ["Matemáticas", "Física"]
}
```

Escalar hacia arriba o hacia afuera

Escalar hacia arriba (scale up)

Se mejora un solo servidor: más CPU, memoria y disco.



Más simple, pero con límite de hardware

Escalar hacia afuera (scale out)

Se agregan más servidores que trabajan en conjunto.



NoSQL: crece agregando nodos

Las bases NoSQL están pensadas para **escalar horizontalmente**: basta con agregar más servidores.

// nosql

Relacional vs. NoSQL

	Relacional	NoSQL
Esquema	Rígido, definido de antemano	Flexible
Modelo	Tablas, filas y columnas	Documentos, grafos, columnas, clave-valor
Consistencia	Transacciones ACID	Algunas sacrifican consistencia (eventual)
Escalabilidad	Normalmente vertical	Horizontal, agregando nodos
Ecosistema	Muy maduro	En crecimiento, menos maduro

Casos de uso NoSQL: aplicaciones web escalables y análisis de datos en tiempo real (p. ej., Elasticsearch).

// cierre

Resumen

- Un **DBMS** gestiona los datos y aporta independencia, integridad, seguridad y concurrencia.
- El **modelo relacional** organiza los datos en tablas unidas por claves primarias y foráneas.
- Los **índices** (árboles B+) aceleran las lecturas de `O(n)` a `O(log n)`, a costa de escrituras y espacio.
- **SQL** define (DDL) y manipula (DML) los datos; los `JOIN` combinan tablas, y los programas (Python, Java) lo usan con consultas **parametrizadas**.
- **Diseñar** es un proceso por fases: conceptual (entidades y relaciones), lógico (tablas y normalización) y físico (SQL e índices).
- **NoSQL** ofrece esquemas flexibles y escalabilidad horizontal.

Referencias: Gillenson, M. (2012). Fundamentals of Database Management Systems. John Wiley & Sons. · Sullivan, M. (2019). NoSQL for Mere Mortals. Addison-Wesley. · Elmasri, R. y Navathe, S. (2016). Fundamentals of Database Systems (7.ª ed.). Pearson.